Guide de Déploiement : RailMind Lite

Ce guide détaille pas à pas la procédure pour installer et déployer la solution complète sur un nouveau serveur ou une nouvelle machine (Windows/Linux) depuis le code source.

---

1. Prérequis Système

Avant de commencer, installez les logiciels suivants sur la machine hôte :

1. **Python 3.10 ou supérieur** (Assurez-vous de cocher "Add Python to PATH" lors de l'installation sous Windows).
2. **Node.js (version 18 ou supérieure)** pour faire tourner l'interface web.
3. **PostgreSQL (version 13 ou supérieure)**.
4. **Ollama** : Téléchargez et installez l'outil depuis ollama.com pour faire tourner l'IA localement.
5. *(Optionnel, mais recommandé)* **Tesseract OCR** : Pour l'extraction de texte sur les PDF scannés.

---

2. Configuration de l'Intelligence Artificielle Locale (Ollama)

1. Ouvrez un terminal (Invite de commandes).
2. Téléchargez et lancez le modèle d'IA requis par l'application :

   ```
   ollama pull llama3.2:3b
   ```

3. Laissez le service Ollama tourner en arrière-plan (généralement, l'application de bureau s'en charge sur Windows). L'API répondra par défaut sur `http://localhost:11434`.

---

3. Configuration de la Base de Données (PostgreSQL)

1. **Pour ouvrir le terminal psql :** Sur Windows, cherchez "SQL Shell (psql)" dans le menu Démarrer (installé par défaut avec PostgreSQL) et ouvrez-le. Appuyez sur Entrée pour accepter les paramètres par défaut jusqu'à ce qu'on vous demande votre mot de passe.
2. Créez une nouvelle base de données nommée `railmind` :

   ```
   CREATE DATABASE railmind;
   ```

3. **Étape cruciale :** Connectez-vous à la base de données `railmind` (tapez `\c railmind`) et activez l'extension Vectorielle pour la recherche sémantique :

   ```
   CREATE EXTENSION IF NOT EXISTS vector;
   ```

---

4. Installation et Démarrage du Backend (FastAPI)

1. Ouvrez un terminal dans le dossier `backend` (le code que vous avez téléchargé).
2. Créez un environnement virtuel Python pour isoler les dépendances :

```
# Sur Windows
python -m venv venv
venv\Scripts\activate
```

3. Installez les librairies requises :

```
pip install -r requirements.txt
```

4. **Configuration :**

   - Le fichier `env.example` est fourni avec le code source dans le dossier `backend`. Copiez-le et renommez la copie en `.env`.
   - Modifiez le contenu du `.env` pour ajouter le mot de passe de la base de données et **la clé API Gemini** :

```
DATABASE_URL=postgresql://postgres:VOTRE_MOT_DE_PASSE@localhost/rai
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b
GEMINI_API_KEY=votre_cle_api_google_ai_studio
JWT_SECRET=une_cle_secrete_tres_longue_et_aleatoire
```

5. Initialisez les tables de la base de données (Toujours dans le terminal du dossier `backend`, avec l'environnement virtuel `venv` activé) :

```
alembic upgrade head
```

6. *(Optionnel)* Insérer les données et documents de démonstration :

```
python seed_demo.py
```

7. **Lancez le serveur Backend :**

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

   L'API tourne désormais en tâche de fond sur ce serveur.

---

5. Installation et Démarrage du Frontend (Next.js)

⚠ **Architecture Serveur/Client :** Ces commandes de démarrage ne doivent être tapées **QUE sur le serveur central** de l'entreprise (ou la machine désignée comme serveur). Les employés n'auront RIEN à installer sur leurs ordinateurs. Ils ouvriront simplement leur navigateur web (Chrome, Edge) et taperont l'adresse IP du serveur (ex: `http://192.168.1.50:3000`) pour utiliser l'application.

1. Ouvrez un **nouveau** terminal dans le dossier `frontend/camrail-rda` (ne coupez pas le terminal du backend).
2. Installez les dépendances Node.js :

```
npm install
```

3. **Configuration :**

- Créez un fichier `.env.local` à la racine de `camrail-rda` avec le contenu suivant (si le serveur a l'IP 192.168.1.50, remplacez `localhost` par cette IP) :

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
```

4. **Lancement de l'interface utilisateur sur le SERVEUR :**

   - **En mode Développement (Tests locaux)** :

     ```
     npm run dev
     ```

   - **En mode Production (Déploiement en entreprise)** :

     ```
     npm run build
     npm run start
     ```

   L'interface web est désormais accessible sur `http://localhost:3000`

---

6. Vérification Finale

Accédez à `http://localhost:3000` via votre navigateur Web.

- Si vous avez exécuté `seed_demo.py`, vous pouvez vous connecter avec le compte `docadmin@camrail.net` (mot de passe : `docadminpassword`).
- Dans le cas contraire, un administrateur SSO devra se connecter pour auto-provisionner le premier compte via Microsoft Azure AD.